A COURSE END PROJECT REPORT ON

STUDENT MARKS MANAGEMENT SYSTEM USING PYTHON

A9511 - PYTHON PROGRAMMING LABORATORY

Submitted by

Thuduri Likith Goud

25881A0518

Course Facilitator

Dr. V. Lokeshwari Vinya

Associate Professor,CSE



Department of Computer Science and Engineering

Vardhaman College of Engineering

Hyderabad

2026-2027

CERTIFICATE

VARDHAMAN COLLEGE OF ENGINEERING

Autonomous Institute, Affiliated to **JNTUH,**
Approved by **AICTE,** Accredited by **NAAC** with **A++** Grade
Kacharam, Shamshabad, Hyderabad- 501 218,
Telangana, India.

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

This is to certify that the Course End Project titled **" Student Marks Management System using Python "** is carried out by " **Thuduri Likith Goud** " with Roll Number " **25881A0518** " towards the **" A9511 – Python Programming Laboratory "** course in partial fulfilment of the requirements for the award of the degree of Bachelor of Technology in Computer Science and Engineering during the Academic year 2026-27.

Course Faculty
Dr. V. Lokeshwari Vinya
Assoc. Prof, CSE

Academic Head
Dr. R. Karthikeyan
Professor, CSE

Head of the Department
Prof. Y. Vijaylatha
Department Of CSE

Contents

Abstract 4
1 Introduction 4
2 Literature Review 4
3 Objectives 5
4 Methodology 6
4.1 Architectural Diagram 6
4.2 Hardware and Software Requirements 6
4.3 Working Principle 7
4.4 Software Implementation 8
5 Results and Discussion 9
6 Mapping POs and SDGs 19
6.1 Program Outcomes (POs) Mapping 19
6.2 Sustainable Development Goals (SDGs) Mapping 19
7 Conclusion 20
Bibliography 21

Abstract

The project “Student Marks Management System using Python” focuses on designing and implementing a system that efficiently stores, manages, and displays student academic records. The system allows users to add, view, search, update, and delete student records in a structured format. Each record contains attributes such as roll number, name, and marks in five subjects (DBMS, DMS, OS, OOPS and BEFA). The core feature of this project is the automatic calculation of total marks, percentage, grade, and pass/fail result for every student.

Student records are held in memory using Python data structures: a list of dictionaries, with a nested dictionary for subject marks. The records are saved permanently in a JSON file, so the data is available every time the program is run. Input validation and exception handling protect the system from invalid data and file errors. The project demonstrates how data structures such as lists, dictionaries and tuples can be combined with file handling to solve a real-world problem in academic record management.

Keywords: Student Marks, Data Structures, Python, List, Dictionary, JSON, File Handling, CRUD Operations, Grade Calculation, Linear Search.

1 Introduction

Student marks management is an essential part of every school, college, and training institute. It involves storing marks, calculating totals and percentages, assigning grades, and declaring results. Doing this manually on paper or in scattered spreadsheets is slow, repetitive, and prone to calculation errors. As the number of students grows, organising and retrieving this data becomes more difficult, which is where data structures and automation play a major role.

This project uses basic Python data structures such as lists, dictionaries, and tuples to store student details and applies a clear grading rule to calculate results automatically. File handling with the JSON format keeps the records permanent. The project also demonstrates algorithmic thinking, an understanding of time complexity, and the practical implementation of searching and record management operations in Python.

2 Literature Review

Lists, dictionaries, searching, and sorting are fundamental topics in data structures and are widely used in real-world applications. Many textbooks explain how arrays, lists, and hash-based structures store and retrieve data, and highlight their efficiency and typical use cases.

Existing student information and examination systems in colleges use databases, web portals, and advanced analytics to store marks, generate reports, and publish results. Basic

implementations use simple lists or files with linear search, while advanced systems use indexed databases and optimised data structures such as hash tables and balanced trees.

This project uses simple built-in data structures and file storage suitable for beginners, focusing on understanding core concepts and implementing them effectively in Python.

3 Objectives

- To design and implement a student marks management system using data structures.
- To apply a grading algorithm that calculates total, percentage, grade, and result automatically.
- To develop a menu-driven program for managing student records (add, view, search, update, delete).
- To validate user input and handle file errors so that only correct data is stored.
- To store the data permanently using JSON file handling.

4 Methodology

4.1 Architectural Diagram

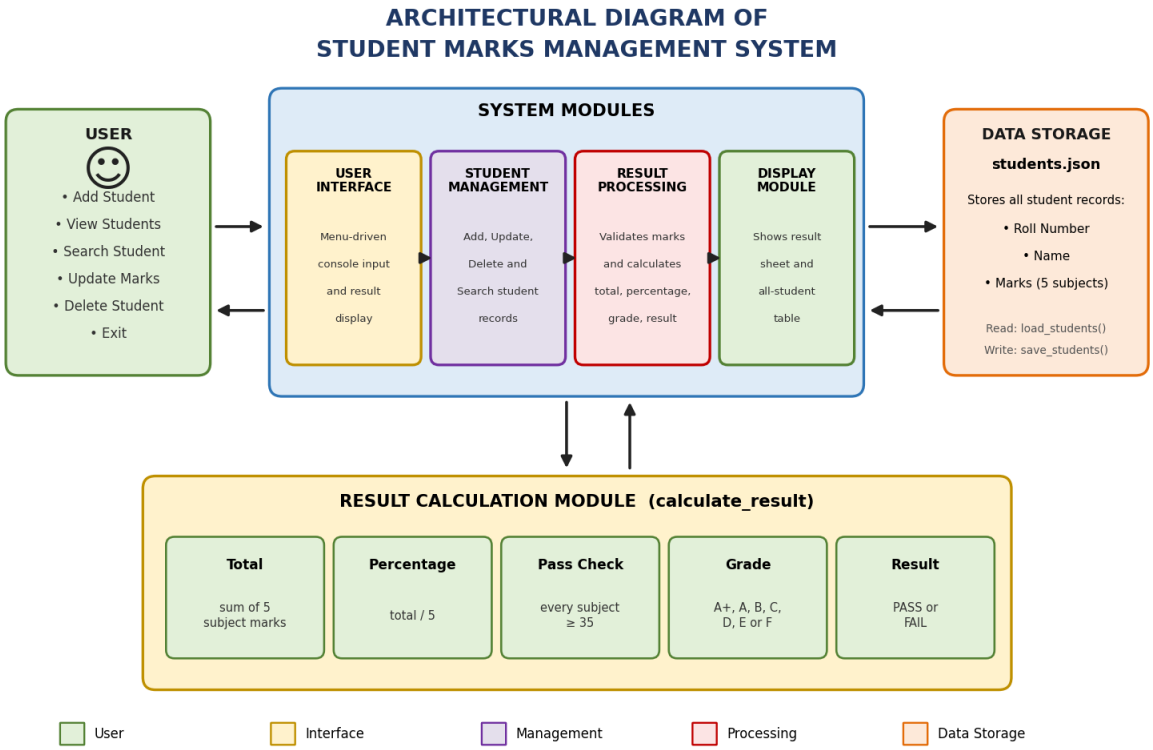


Figure 1: Architectural Diagram of Student Marks Management System

The system consists of a user interface, processing logic, and a data storage unit. The user enters student details and marks, which are stored in a data structure. The result calculation module processes the marks and the display module shows the formatted result based on the selected operation.

4.2 Hardware and Software Requirements

Hardware Requirements:

- Computer/Laptop with minimum 4GB RAM
- Processor: Intel i3 or above
- Keyboard and monitor

Software Requirements:

- Programming Language: Python 3.6 or higher
- Libraries: json and os (Python standard library, no installation required)
- Editor/IDE: IDLE / VS Code / PyCharm
- Operating System: Windows/Linux/macOS

4.3 Working Principle

The system stores student details in a list of dictionaries.

- Users can perform operations like add, view, search, update, and delete student records.
- Each record is a dictionary with the keys roll, name, and marks. The marks are a nested dictionary that maps each subject to its mark.
- The function calculate_result() computes the total, percentage, grade, and result and returns them as a tuple.
- A student passes only if every subject mark is at least 35. Otherwise, the grade is F and the result is FAIL.
- The program uses menu-driven logic for user interaction, and every change is saved to students.json.

Data Structures Used:

Data Structure	Where it is used	Purpose
List	students: holds all student records	Keeps records in order; easy to append, iterate, and remove

Data Structure	Where it is used	Purpose
Dictionary	Each student: roll, name, marks	Accesses data by field name instead of position
Nested Dictionary	marks: subject → mark	Works directly with sum() and all() for calculations
List (constant)	SUBJECTS	Fixes the subject order; subjects can be changed in one place
Tuple	Return value of calculate_result()	Returns total, percentage, grade, and result together

Grading Rules:

Percentage = Total marks / 5 (each subject is out of 100). A student must score at least 35 in every subject to pass.

Percentage	Grade	Result
90 and above	A+	PASS
80 to 89.99	A	PASS
70 to 79.99	B	PASS
60 to 69.99	C	PASS
50 to 59.99	D	PASS
Below 50 (all subjects 35 or above)	E	PASS
Any subject below 35	F	FAIL

4.4 Software Implementation

Algorithm Steps:

1. Start
2. Load existing student records from students.json (empty list if the file is missing)
3. Display menu (Add, View, Search, Update, Delete, Exit)
4. If Add → Read roll number, name, and marks; reject duplicates; save the record
5. If View → Display all students with total, percentage, grade, and result
6. If Search → Find the student by roll number and display the result
7. If Update → Find the student by roll number and re-enter the marks
8. If Delete → Find the student by roll number, confirm, and remove the record
9. Repeat until Exit

Pseudo Code:

```
START
LOAD list students FROM file
WHILE true DO
  DISPLAY menu
  READ choice
  IF choice == 1 THEN
    ADD student
  ELSE IF choice == 2 THEN
    DISPLAY all students
  ELSE IF choice == 3 THEN
    SEARCH student
  ELSE IF choice == 4 THEN
    UPDATE student marks
  ELSE IF choice == 5 THEN
    DELETE student
  ELSE IF choice == 6 THEN
    EXIT
  ELSE
    DISPLAY "Invalid choice"
END WHILE
END
```

5 Results and Discussion

Implementation Overview:

The system successfully manages student marks and calculates results automatically. The program is implemented using Python and demonstrates effective use of lists, dictionaries, tuples, and JSON file handling.

CODE:

```
import json
import os


DATA_FILE = "students.json"


SUBJECTS = [
  "DBMS",
  "DMS",
  "OS",
  "OOPS",
  "BEFA"
]


PASS_MARK = 35



# Load student records from the JSON file
def load_students():
  if not os.path.exists(DATA_FILE):
    return []


  try:
    with open(DATA_FILE, "r") as file:
```

```
        return json.load(file)
    except (json.JSONDecodeError, OSError):
        print("Warning: Could not read student data.")
        return []



# Save student records to the JSON file
def save_students(students):
    try:
        with open(DATA_FILE, "w") as file:
            json.dump(students, file, indent=4)
    except OSError:
        print("Error: Unable to save student records.")



# Read and validate integer input
def get_integer(prompt, minimum, maximum):
    while True:
        try:
            value = int(input(prompt))

            if minimum <= value <= maximum:
                return value

            print(f"Enter a value between {minimum} and {maximum}.")

        except ValueError:
            print("Invalid input. Enter a whole number.")



# Calculate total, percentage, grade, and result
def calculate_result(marks):
    total = sum(marks.values())
    percentage = total / len(SUBJECTS)

    passed = all(marks[s] >= PASS_MARK for s in SUBJECTS)

    if not passed:
        grade = "F"
        result = "FAIL"
    else:
        result = "PASS"

        if percentage >= 90:
            grade = "A+"
        elif percentage >= 80:
            grade = "A"
        elif percentage >= 70:
            grade = "B"
        elif percentage >= 60:
            grade = "C"
        elif percentage >= 50:
            grade = "D"
        else:
            grade = "E"

    return total, percentage, grade, result
```

```
# Display one student's result
def display_student(student):
    total, percentage, grade, result = calculate_result(
        student["marks"]
    )

    print("\n" + "=" * 45)
    print("       STUDENT RESULT")
    print("=" * 45)
    print("Roll Number :", student["roll"])
    print("Name       :", student["name"])

    for subject in SUBJECTS:
        print(f"{subject:<20}: {student['marks'][subject]}")

    print("-" * 45)
    print(f"Total      : {total}/{len(SUBJECTS) * 100}")
    print(f"Percentage  : {percentage:.2f}%")
    print("Grade      :", grade)
    print("Result     :", result)
    print("=" * 45)



# Add a new student
def add_student(students):
    print("\n--- Add Student ---")

    roll = input("Enter roll number: ").strip()

    if not roll:
        print("Roll number cannot be empty.")
        return

    if any(s["roll"].lower() == roll.lower() for s in students):
        print("A student with this roll number already exists.")
        return

    name = input("Enter student name: ").strip()

    if not name:
        print("Name cannot be empty.")
        return

    marks = {}

    for subject in SUBJECTS:
        marks[subject] = get_integer(
            f"Enter {subject} marks (0-100): ", 0, 100
        )

    student = {
        "roll": roll,
        "name": name,
        "marks": marks
    }
```

```
    students.append(student)
    save_students(students)

    print("Student added successfully!")
    display_student(student)



# Display all student records
def view_students(students):
    if not students:
        print("No student records found.")
        return

    print("\n--- All Student Records ---")
    print(
        f"{'Roll':<12}{'Name':<20}"
        f"{'Total':<10}{'Percent':<12}{'Grade':<8}{'Result'}"
    )
    print("-" * 75)

    for student in students:
        total, percentage, grade, result = calculate_result(
            student["marks"]
        )

        print(
            f"{student['roll']:<12}"
            f"{student['name'][:19]:<20}"
            f"{total:<10}"
            f"{percentage:.2f}%{'':<5}"
            f"{grade:<8}{result}"
        )



# Search for a student
def search_student(students):
    roll = input("Enter roll number to search: ").strip()

    for student in students:
        if student["roll"].lower() == roll.lower():
            display_student(student)
            return

    print("Student not found.")



# Update an existing student's marks
def update_marks(students):
    roll = input("Enter roll number to update: ").strip()

    for student in students:
        if student["roll"].lower() == roll.lower():
            print("Enter the updated marks:")

            for subject in SUBJECTS:
                student["marks"][subject] = get_integer(
```

```
            f"{subject} (0-100): ", 0, 100
          )

        save_students(students)
        print("Marks updated successfully!")
        display_student(student)
        return

  print("Student not found.")



# Delete a student record
def delete_student(students):
  roll = input("Enter roll number to delete: ").strip()

  for student in students:
    if student["roll"].lower() == roll.lower():
      confirm = input(
        f"Delete {student['name']}'s record? (yes/no): "
      ).strip().lower()

      if confirm == "yes":
        students.remove(student)
        save_students(students)
        print("Student record deleted.")
      else:
        print("Deletion cancelled.")

      return

  print("Student not found.")



# Main menu
def main():
  students = load_students()

  while True:
    print("\n===== STUDENT MARKS MANAGEMENT SYSTEM =====")
    print("1. Add Student")
    print("2. View All Students")
    print("3. Search Student")
    print("4. Update Student Marks")
    print("5. Delete Student")
    print("6. Exit")

    choice = input("Enter your choice (1-6): ").strip()

    if choice == "1":
      add_student(students)
    elif choice == "2":
      view_students(students)
    elif choice == "3":
      search_student(students)
    elif choice == "4":
      update_marks(students)
    elif choice == "5":
```

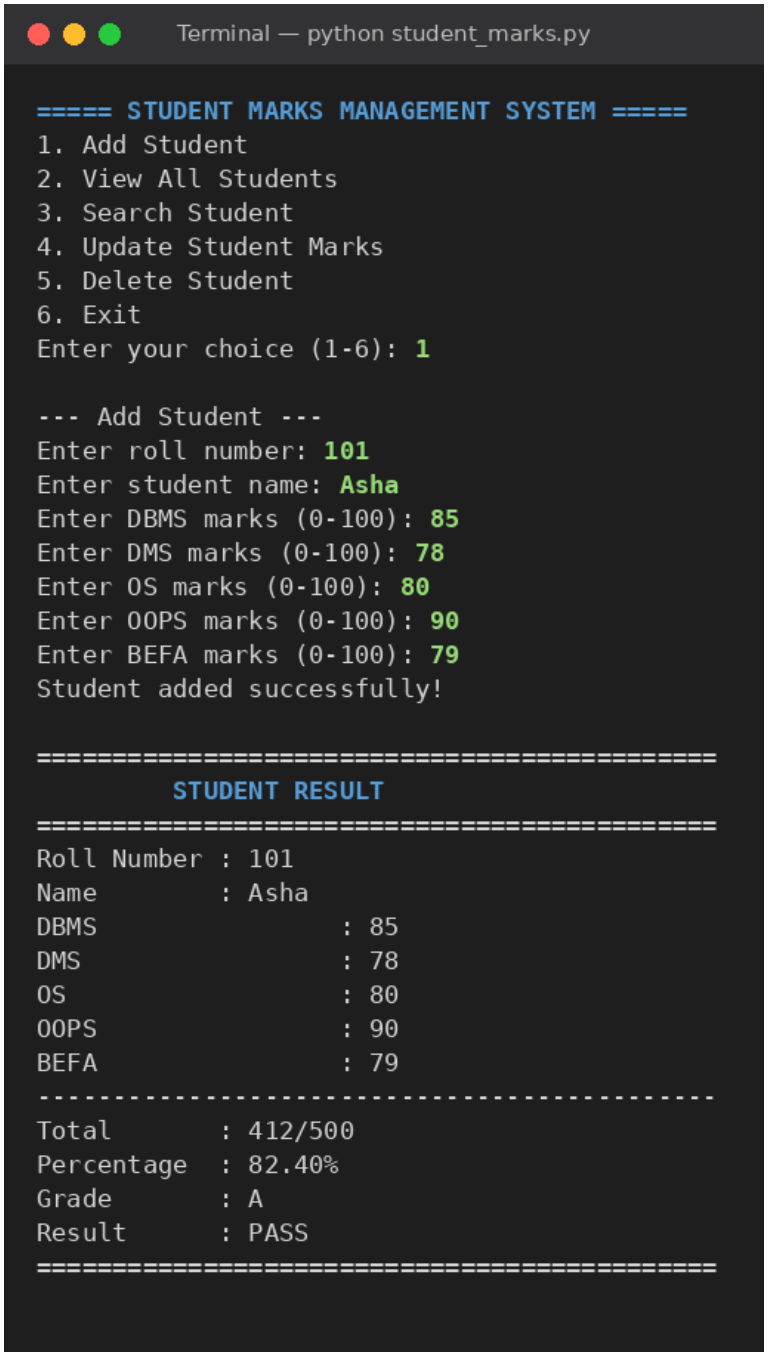
```
        delete_student(students)
    elif choice == "6":
        print("Thank you for using the system!")
        break
    else:
        print("Invalid choice. Please select 1-6.")



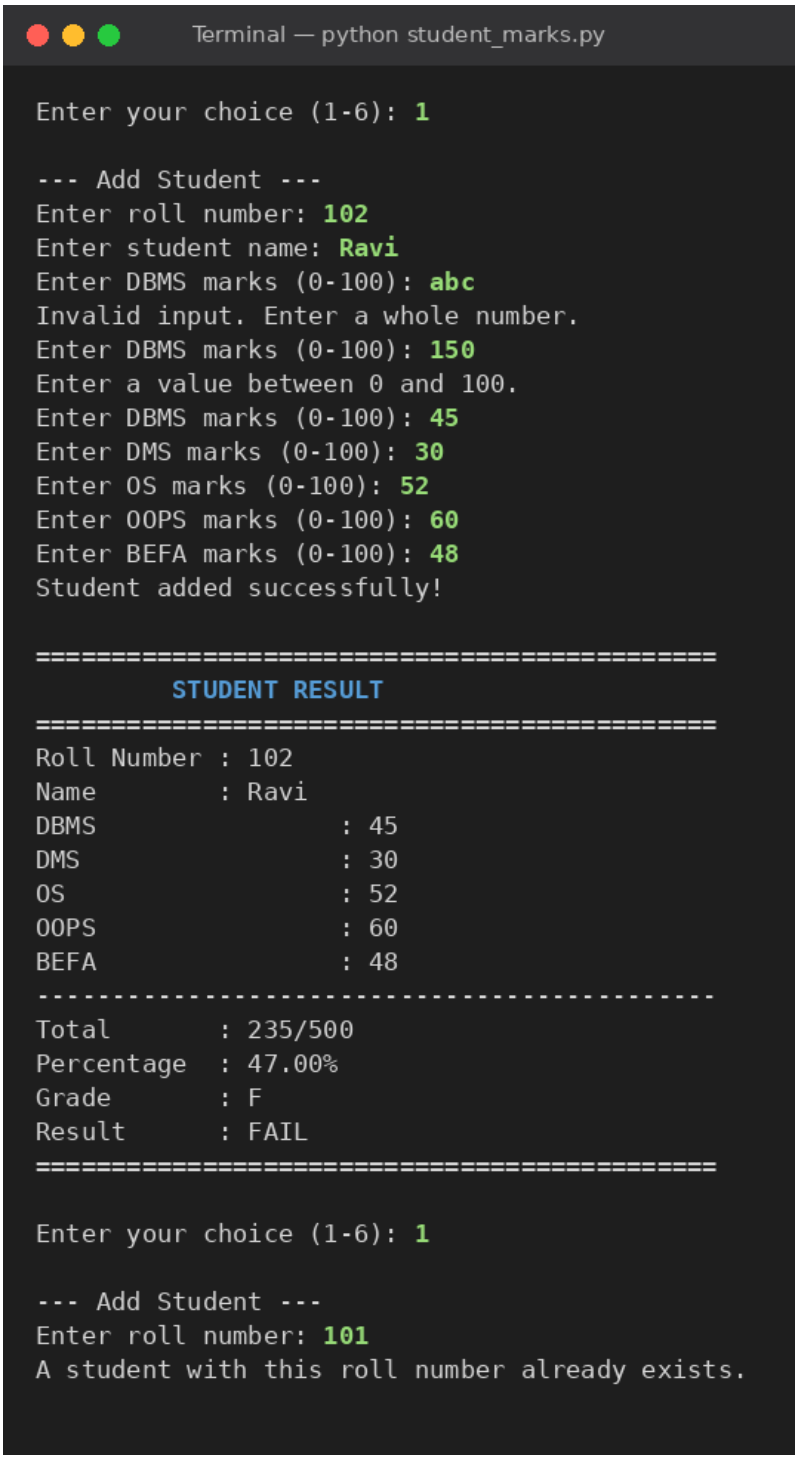
if __name__ == "__main__":
    main()
```

OUTPUT:

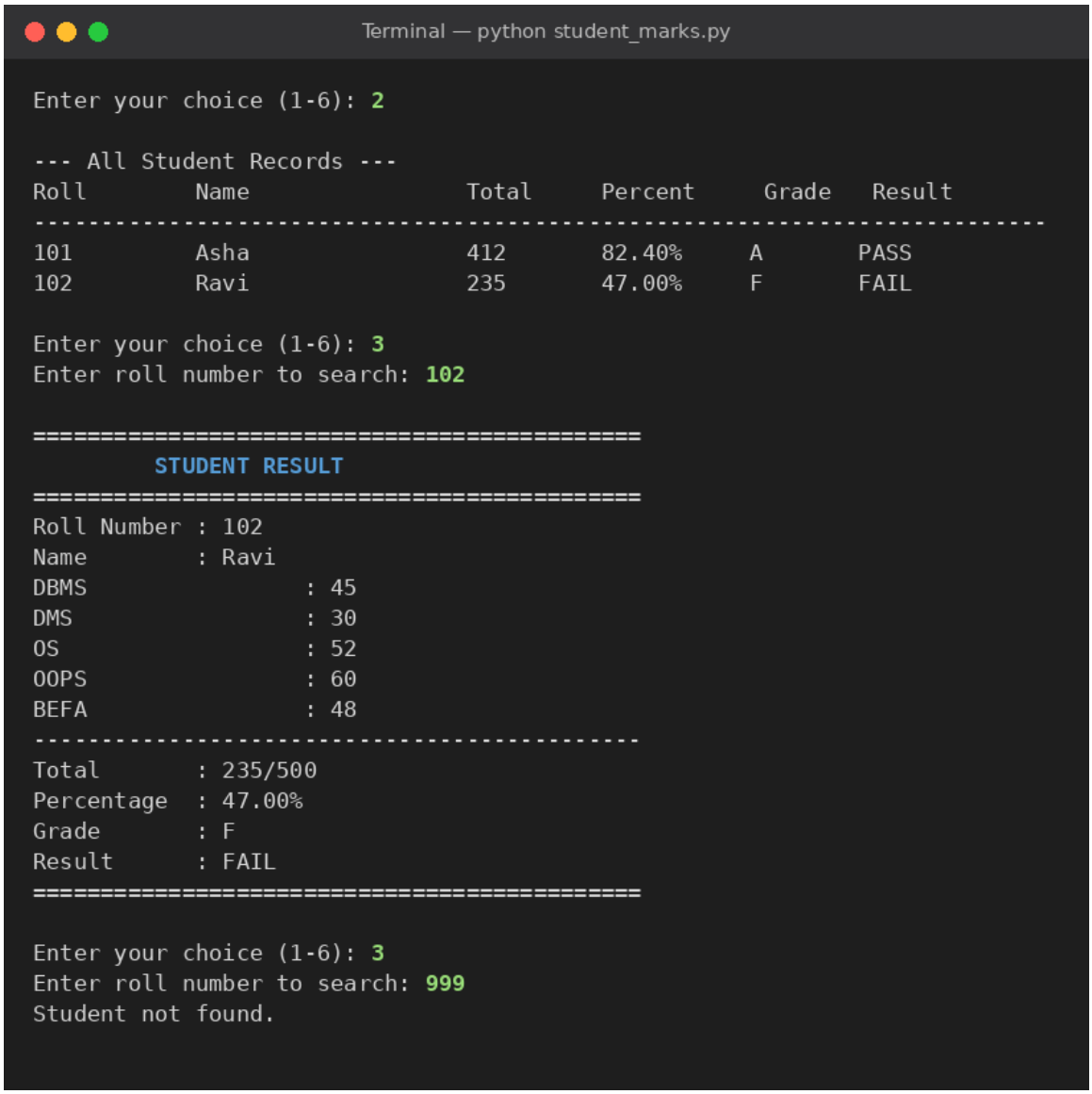
Output 1: Adding a student and displaying the result sheet



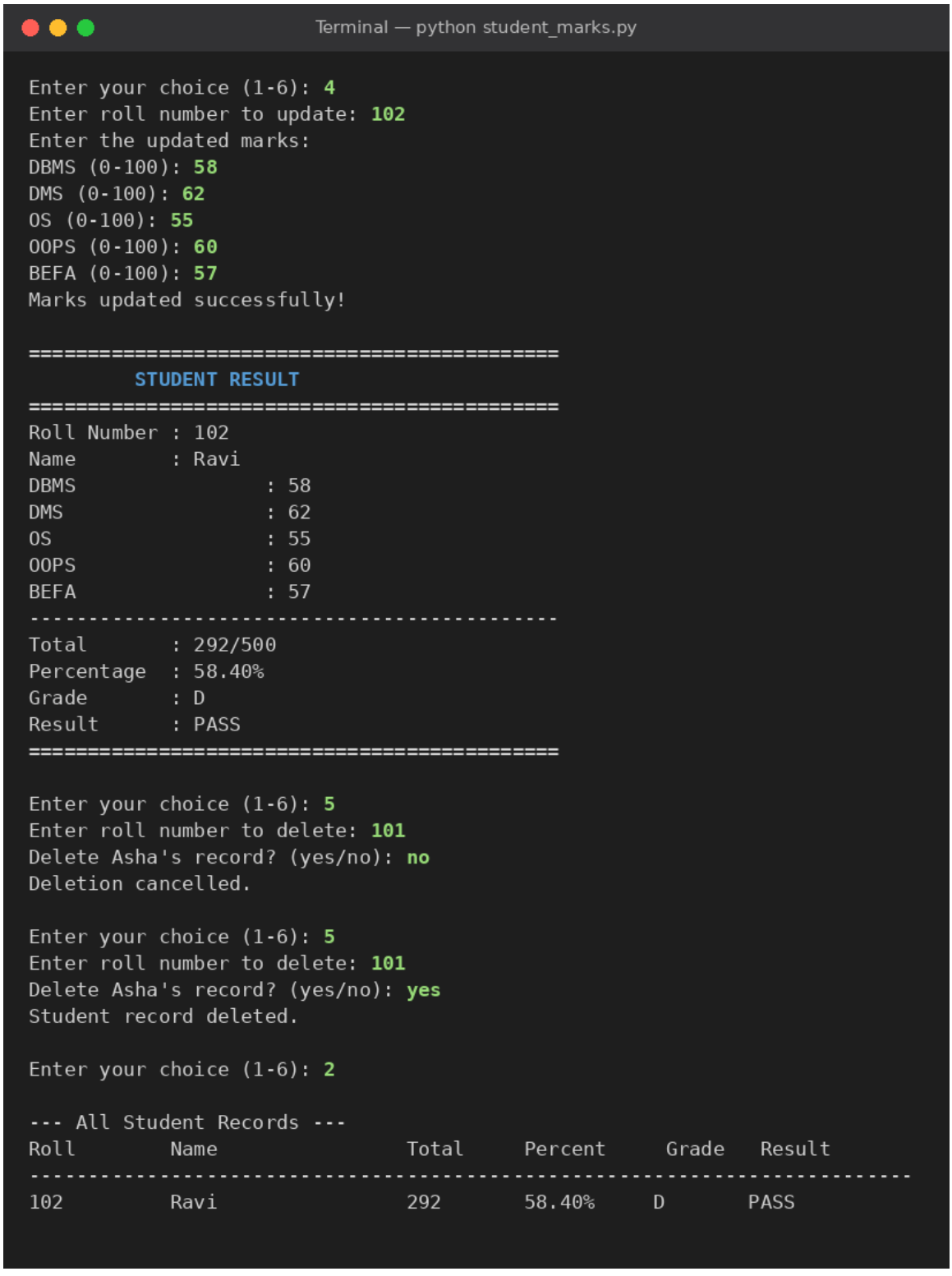
Output 2: Input validation, a FAIL result, and a duplicate roll number



Output 3: Viewing all students and searching by roll number



Output 4: Updating marks and deleting a record



Time Complexity:

Operation	Complexity	Reason
Add student	O(n)	Duplicate check scans all n records; the append itself is O(1)
View all	O(n)	Visits every record once
Search	O(n)	Linear search by roll number
Update	O(n)	Linear search, then a fixed number of subjects
Delete	O(n)	Linear search and list removal
Save / Load file	O(n)	The whole list is written or read

Summary of Key Results:

The project efficiently stores, calculates, and displays student result data based on user input. Invalid marks, empty fields, and duplicate roll numbers are rejected, and the records are preserved in students.json between runs.

Interpretation:

Automatic calculation removes manual errors, and structured storage makes records quick to find, update, and display, making the system user-friendly.

Achievement of Objectives:

- Student marks management system implemented successfully.
- Grading algorithm applied effectively.
- Menu-driven interface developed.
- Input validation and file storage implemented.

Implications:

This system can be extended to real-world applications such as college examination systems, school result portals, and online learning platforms.

6 Mapping POs and SDGs

6.1 Program Outcomes (POs) Mapping

Table 1: Mapping of Program Outcomes (POs) to Project Relevance

PO No.	Program Outcome	Relevance
PO1	Engineering Knowledge	Applied data structures (list, dictionary, tuple) and a grading algorithm
PO2	Problem Analysis	Solved the problem of organising and evaluating student marks
PO3	Design/Development of Solutions	Developed a menu-driven student marks system
PO4	Conduct Investigations of Complex Problems	Tested the system with valid, invalid, and boundary inputs
PO5	Engineering Tool Usage	Used Python and the json and os modules

6.2 Sustainable Development Goals (SDGs) Mapping

Table 2: Mapping of Sustainable Development Goals (SDGs) to Project Relevance

SDG No.	Goal	Relevance
SDG 4	Quality Education	Helps institutions track student performance and helps students learn data structures
SDG 9	Industry, Innovation and Infrastructure	Supports digital academic record management
SDG 12	Responsible Consumption and Production	Reduces paper-based record keeping

7 Conclusion

The project successfully demonstrates the implementation of a Student Marks Management System using Python data structures and file handling. It highlights how lists, dictionaries, and tuples can be used to manage and organise data efficiently. Automatic result calculation, input validation, and permanent storage improve accuracy and usability.

Future Work:

- Add sorting algorithms (such as Merge Sort or Quick Sort) to rank students and list toppers
- Use a dictionary keyed by roll number for O(1) searching
- Use database integration (SQLite/MySQL)
- Implement a GUI or web interface
- Add class statistics such as average, highest, and lowest marks

Bibliography

- Python Software Foundation – Python 3 Documentation (docs.python.org)
- GeeksforGeeks – Python Data Structures and File Handling
- TutorialsPoint – Data Structures
- Python json module documentation
- Research articles on student information systems and academic record management